Data 101/CS 187: Data Engineering Fall 2026 Discussion 6 – October 16th, 2026

Window Functions, Data Granularity

Instructor: Aditya Parameswaran *Covers Lecture(s): 11 – 12*

I Window Functions

1. Let's revisit a fictional scenario where instructors order boba for every enrolled Data 101 student. In the spirit of boosting morale during midterm season, the instructors have decided to allow multiple boba orders per student. Order IDs are assigned in the same order that orders are received.

   - `orders (student_id, student_name, order_id, order_info, price)`

   To keep track of the spirited spending, we need to calculate the cumulative total spent by each student up to and including each of their orders. Write a SQL query that returns the student's name, order ID, and their cumulative total spend from the orders table. Make sure the orders are returned in ascending order of `student_name` and `order_id`.

2. Let's also track how much the students' order price compares with a previous order. For each order in `orders` table, find the price of the previous order placed by the same student. Show the student's name, the order ID, the current order's price, and the previous order's price. Sort in ascending order of the student's name and then order ID.

3. Assume that we can process at most 10 orders at a time. For each order in the `orders` table, compute the sum of prices of all orders whose `order_id` is 10 before the current order's `order_id` (i.e., current order and previous orders with order_id difference $\leq$ 10), including the current order. Show the student's name, the order ID, and a column called `rolling_sum_10_orders` showing this total. Return the results ordered by `order_id`.

4. In Project 1, we worked with a `movie_sample` table to explore gross earnings for each movie and stored the results in a view called `movie_gross`. Let's explore how to leverage window functions to compute the average earnings **per genre** across movies, as well as the movie's rank within its genre based on gross earnings.

id	movie_id	info_type_id	info
6215794	2022105	3	Comedy
6890556	2441099	3	Drama
1457358	1924463	107	$22,946,000 (USA) (25 December 2005)

Table 1: Sample rows of `movie_info_sample`.

gross	movie_id	title
760507625.0	1704289	Avatar
658672302.0	2438179	Titanic
623357910.0	2346436	The Avengers

Table 2: Sample rows of `movie_gross`

Create a new view called `movie_avg_genre` with the following columns: `movie_id`, `title`, `gross`, `genre`, `avg_gross_genre`, `genre_rank`. Following the view definition, write a `SELECT` query to return the rows for the movie 'Mr. & Mrs. Smith' ordered by genre alphabetically.

- `gross`: The gross earnings of the given movie
- `avg_gross_genre`: The *average* gross earnings of movies with the same genre as the given movie. If a movie belongs to multiple genres, it should appear in multiple rows, with each row corresponding to a different genre.
- `genre_rank`: The rank of each movie within its genre based on gross earnings (highest gross = 1). If two movies have the same gross earnings, they should share the same rank and skip the next rank. For example, if Avatar and Titanic were in the same genre and were ranked 1, the next rank available would be rank 3.

Hint: Check out the movie genre info type, with `info_type_id = 3` from `movie_info_sample`.

5. Expanding on the previous question, besides finding the average gross for movies in each genre but also finding the total gross for each genre.

   Create a view called `movie_avg_total_genre` with the following columns: `movie_id`, `title`, `gross`, `genre`, `avg_gross`, `total_gross`.

   - `gross`: The gross earnings of the given movie
   - `avg_gross`: The average gross earnings of movies with the same genre as the given movie. If a movie belongs to multiple genres, it should appear in multiple rows, with each row corresponding to a different genre.
   - `total_gross`: The total gross earning of movies with the same genre as the given movie.

Bonus: Instead of writing out two separate windows, is it possible to simplify?

II Data Granularity: Walking a Hierarchy of Arbitrary Depth

In this section, we will work with a table with a complex hierarchy: the taxonomic hierarchy of biology from US National Center for Biotechnology Information. The relevant schema for this exercise is as follows and note that `(tax_id, rank)` is what makes a unique node in the table:

- `nodes (tax_id INTEGER, parent INTEGER, rank TEXT`)
- `names (tax_id INTEGER, name_txt TEXT)`

6. Write a SQL query to find the node representing the *Animalia* kingdom. Your query should return the kingdom name and its corresponding node information.

7. Let us drill down into the *Animalia* kingdom. First, find all children nodes of the *Animalia* kingdom (ID 33208). Your query should return the `tax_id` and `rank` of each child node and their parent. Hint: use self-join.

8. Next, find the names of these children nodes, along with the names of their parents *Animalia*. Your query should return the `tax_id`, `rank`, and `name_txt` of each child and parent node.

9. In the biological taxonomy data, it is common for the same phylum to have multiple synonymous names, all sharing the same `tax_id`. Write a SQL query to return the parent node (Animalia) and its direct child nodes. For each child and parent node, aggregate all its synonymous names into a single JSON array.

10. **[Optional, Challenge]** How can we drill down one more layer? What if we want to get the names of all the classes under the *Animalia* kingdom?